LATENT VARIABLES, MATRIX FACTORISATION AND MCMC PROBLEM SHEET

This sheet covers the lectures on probabilistic inference, matrix factorisation and MCMC. Question 1 practises generative and hierarchical models and the EM algorithm, Question 2 low-rank approximation and matrix completion, Question 3 drawing random samples, and Question 4 Markov chains and the Metropolis algorithm. Each ends with a short Python part (`import numpy as np` is assumed). Question 5 is harder and optional: it proves that EM and MCMC work, and runs alternating least squares.

1

(a) An email is spam with probability 0.4. The word "free" appears in 30% of spam emails and in 2% of other emails. Write down the joint distribution of S (spam or not) and F (contains "free" or not) as a 2×2 table. Find $\mathbb{P}(F = \text{yes})$ and $\mathbb{P}(S = \text{spam} \mid F = \text{yes})$.

Using $\mathbb{P}(S,F) = \mathbb{P}(F \mid S)\mathbb{P}(S)$:

	$F = \text{yes}$	$F = \text{no}$
$S = \text{spam}$	$0.4 \times 0.3 = 0.12$	$0.4 \times 0.7 = 0.28$
$S = \text{not spam}$	$0.6 \times 0.02 = 0.012$	$0.6 \times 0.98 = 0.588$

Marginalising over S, $\mathbb{P}(F = \text{yes}) = 0.12 + 0.012 = 0.132$, so $\mathbb{P}(S = \text{spam} \mid F = \text{yes}) = 0.12/0.132 \approx 0.91$. We modelled how emails are generated (a generative model) and used it to classify (discriminate).

[3 marks]

(b) In the hierarchical ratings model of the lecture, $\theta_j \sim \text{Beta}(a,b)$ and $k_j \sim \text{Bin}(n_j,\theta_j)$, the posterior mean of θ_j is $(a + k_j)/(a + b + n_j)$. Take $a = 20$, $b = 5$. Product C has 9 positive reviews out of 10, and product D has 85 out of 100. Rank them by the fraction of positive reviews and by the posterior mean. How many of its 10 reviews would C need to be positive to rank above D?

By fraction C (0.90) beats D (0.85). By posterior mean C gets $29/35 \approx 0.829$ and D gets $105/125 = 0.840$, so D ranks higher. C needs $(20 + k)/35 > 0.84$, i.e. $k > 9.4$: all 10 reviews positive (giving $30/35 \approx 0.857$). The prior is worth $a + b = 25$ reviews at the typical rate 0.8, which outweighs C's ten reviews but not D's hundred.

[4 marks]

(c) A latent variable Z is 1 or 0 with probability $\frac{1}{2}$ each. Given $Z = 1$, $X \sim \mathcal{N}(0,1)$; given $Z = 0$, $X \sim \mathcal{N}(4,1)$. Write down the density $f(x)$ of X. Show that the *responsibility* $\gamma(x) = \mathbb{P}(Z = 1 \mid X = x)$ is $1/(1 + \mathrm{e}^{4x-8})$, and evaluate it at $x = 1$, 2 and 3.

Marginalising over Z, $f(x) = \frac{1}{2}\mathcal{N}(x \mid 0,1) + \frac{1}{2}\mathcal{N}(x \mid 4,1)$. By Bayes' rule (the $\frac{1}{2}$s and the normalisations $1/\sqrt{2\pi}$ cancel)

$$\gamma(x) = \frac{\mathrm{e}^{-x^2/2}}{\mathrm{e}^{-x^2/2} + \mathrm{e}^{-(x-4)^2/2}} = \frac{1}{1 + \mathrm{e}^{(x^2-(x-4)^2)/2}} = \frac{1}{1 + \mathrm{e}^{4x-8}}.$$

$\gamma(1) \approx 0.982$, $\gamma(2) = 0.5$ and $\gamma(3) \approx 0.018$. This is the logistic sigmoid of a linear function of x: a generative model with equal variances leads to logistic regression.

[3 marks]

(d) Take the model of part (c) but treat $p = \mathbb{P}(Z=1)$, μ_A and μ_B as unknown (the variances stay 1). The data are $X = (-1, 0, 1, 2, 4, 5)$. Starting from $p = \frac{1}{2}$, $\mu_A = 0$, $\mu_B = 4$, carry out one step of EM: compute the responsibilities γ_i (use part (c)), then the new values

$$p = \frac{1}{m}\sum_{i=1}^{m}\gamma_i, \qquad \mu_A = \frac{\sum_i \gamma_i X_i}{\sum_i \gamma_i}, \qquad \mu_B = \frac{\sum_i (1-\gamma_i) X_i}{\sum_i (1-\gamma_i)}.$$

From part (c), $\gamma \approx (1.000, 1.000, 0.982, 0.500, 0.000, 0.000)$ (more precisely 0.99966 for $X=0$ and 0.00034 for $X=4$). Then $\sum_i \gamma_i \approx 3.482$, so $p \approx 3.482/6 \approx 0.58$, and

$$\mu_A \approx \frac{-1+0+0.982+1.0+0.001}{3.482} \approx 0.28, \quad \mu_B \approx \frac{0.018+1.0+4.0+5}{2.518} \approx 3.98.$$

The point $X=2$ sits midway, so half of it is given to each cluster: EM makes *soft* assignments.

[4 marks]

(e) In the M-step the terms of Q that involve p are $\sum_{i=1}^{m}\big(\gamma_i \log(p) + (1-\gamma_i)\log(1-p)\big)$. Show that they are maximised by $p = \frac{1}{m}\sum_i \gamma_i$.

Setting the derivative to zero, $\frac{1}{p}\sum_i \gamma_i - \frac{1}{1-p}\sum_i (1-\gamma_i) = 0$, so $(1-p)\sum_i \gamma_i = p(m - \sum_i \gamma_i)$, giving $p = \frac{1}{m}\sum_i \gamma_i$: the average responsibility. (The second derivative is negative, so this is a maximum.)

[2 marks]

(f) In Python, generate $m = 200$ points from the model with $p = 0.3$, $\mu_A = 0$, $\mu_B = 4$, start EM from $p = 0.5$, $\mu_A = -1$, $\mu_B = 1$, and print the log-likelihood $\sum_i \log\big(p\mathcal{N}(X_i \mid \mu_A, 1) + (1-p)\mathcal{N}(X_i \mid \mu_B, 1)\big)$ at each iteration. Does it ever decrease?

```
rng = np.random.default_rng(0)
m = 200
Z = rng.random(m) < 0.3
X = np.where(Z, rng.normal(0, 1, m), rng.normal(4, 1, m))
def N(x, mu):
    return np.exp(-(x - mu)**2 / 2) / np.sqrt(2*np.pi)
p, muA, muB = 0.5, -1.0, 1.0
for t in range(20):
    a, b = p * N(X, muA), (1 - p) * N(X, muB)
    print(t, np.sum(np.log(a + b)))
    g = a / (a + b)    # E-step: responsibilities
    p = g.mean()    # M-step
    muA = g @ X / g.sum()
    muB = (1 - g) @ X / (1 - g).sum()
print(p, muA, muB)
```

The log-likelihood goes -1056.4, -407.9, -379.2, -377.46, -377.37, …, and has settled at -377.363 by iteration 9. It never decreases (once converged it wobbles by about 10^{-13}, which is rounding error). The estimates are $p = 0.274$, $\mu_A = 0.010$ and $\mu_B = 4.050$: close to the truth, and closer still to this sample, in which 26.5% of the points are of type A.

[4 marks]

End of question 1

Answers

2

(a) A film site has 1000 users and 500 films. How many numbers does a rank $K = 20$ factorisation **AB** of the ratings matrix have, compared with the full matrix? LoRA adds a rank $K = 4$ update to a 1024×1024 weight matrix: what fraction of the weights does it train?

$(1000 + 500) \times 20 = 30000$ against $1000 \times 500 = 500000$, so 6%. For LoRA, $(1024 + 1024) \times 4 = 8192$ against $1024^2 = 1048576$: about 0.8%.

[3 marks]

(b) For the ratings in the lecture

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

use $R_{ui} \approx \bar{R}_u + \bar{R}_i - \bar{R}$ to predict Bob's rating of The Matrix and Chas's rating of Titanic.

The overall mean of the 11 ratings is $\bar{R} = 37/11 \approx 3.36$. Bob: $\bar{R}_u = 11/3 \approx 3.67$ and The Matrix has $\bar{R}_i = 3.5$, giving $3.67 + 3.5 - 3.36 \approx 3.80$. Chas: $\bar{R}_u = 13/4 = 3.25$ and Titanic has $\bar{R}_i = 3.5$, giving ≈ 3.39. (Bob rates generously, Chas less so.)

[4 marks]

(c) The matrix $\mathbf{X} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ is symmetric with positive eigenvalues, so its SVD is its eigen-decomposition. Find it, write down the best rank-1 approximation $\mathbf{X}_1$, and check that $\|\mathbf{X} - \mathbf{X}_1\|_F$ equals the discarded singular value, as Eckart–Young says.

The eigenvectors are $\boldsymbol{u}_1 = (1,1)^\mathsf{T}/\sqrt{2}$ with eigenvalue $s_1 = 4$ and $\boldsymbol{u}_2 = (1,-1)^\mathsf{T}/\sqrt{2}$ with $s_2 = 2$. Then

$$\mathbf{X}_1 = s_1 \boldsymbol{u}_1 \boldsymbol{u}_1^\mathsf{T} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}, \qquad \mathbf{X} - \mathbf{X}_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix},$$

and $\|\mathbf{X} - \mathbf{X}_1\|_F = \sqrt{1+1+1+1} = 2 = s_2$. (The spectral norm of the error is also 2.)

[4 marks]

Answers

(d) In alternating least squares with $K = 1$ every factor is a number. A user has rated three films whose factors are $b_j = 1$, 2 and 0.5, giving ratings $X_j = 2$, 4 and 1. With the item factors fixed, the user's factor is the ridge regression solution $a = \sum_j b_j X_j / \left(\eta + \sum_j b_j^2\right)$. Find a and the predicted rating of a fourth film with $b = 1.5$ for $\eta = 0$, 0.5 and 5. Comment.

$\sum_j b_j X_j = 2 + 8 + 0.5 = 10.5$ and $\sum_j b_j^2 = 5.25$.

η	0	0.5	5
a	2.000	1.826	1.024
prediction $1.5a$	3.00	2.74	1.54

With $\eta = 0$ the ratings are fitted exactly ($X_j = 2b_j$). The regulariser (the prior) shrinks a towards 0, more strongly when there are few ratings. Shrinking towards a rating of 0 is a poor default, which is why in practice we first subtract means such as those of part (b) and factorise what is left.

[4 marks]

(e) In Python, make an 8×6 matrix of rank 2, hide about a quarter of its entries, and fill them in with the iterative SVD method of the lecture (start from the user means; keep the best rank-2 approximation; copy its values into the missing entries only). Print the RMS error on the missing entries as you go.

```
rng = np.random.default_rng(0)
F = rng.integers(1, 4, (8, 2))
X = F @ rng.integers(0, 3, (2, 6))    # rank 2
W = rng.random(X.shape) > 0.25    # True where observed
miss = np.logical_not(W)
Xt = np.where(W, X, 0.0)
mean = Xt.sum(1) / W.sum(1)    # user means
Xt = np.where(W, X, mean[:, None])
for t in range(201):
    if t in (0, 1, 5, 20, 50, 100, 200):
        print(t, np.sqrt(np.mean((Xt - X)[miss]**2)))
    U, s, Vt = np.linalg.svd(Xt)
    L = (U[:, :2] * s[:2]) @ Vt[:2]    # rank 2
    Xt = np.where(W, X, L)
```

Here 12 of the 48 entries are missing. The RMS error is 2.79 with the user means, then 2.04, 1.26, 0.67, 0.14, 0.011 and 8×10^{-5} after 1, 5, 20, 50, 100 and 200 iterations. Because the matrix really has rank 2 the missing entries are recovered exactly, but convergence is slow.

[5 marks]

End of question 2

3

(a) We estimate $I = \int_0^1 \mathrm{e}^x \,\mathrm{d}x = \mathbb{E}\left[\mathrm{e}^U\right]$, with U uniform on $(0,1)$, by averaging e^{U_i} over n independent samples. Find $\mathrm{Var}(\mathrm{e}^U)$, the standard error for $n = 10^4$, and the n needed for a standard error of 10^{-4}.

$I = \mathrm{e} - 1 \approx 1.718$ and $\mathbb{E}\left[\mathrm{e}^{2U}\right] = (\mathrm{e}^2 - 1)/2 \approx 3.195$, so $\mathrm{Var}(\mathrm{e}^U) = 3.195 - 1.718^2 \approx 0.242$. The standard error is $\sqrt{0.242/n}$: 0.0049 for $n = 10^4$. For 10^{-4} we need $n = 0.242/10^{-8} \approx 2.4 \times 10^7$. The error falls only as $1/\sqrt{n}$, so each extra decimal place costs 100 times as many samples.

[4 marks]

(b) The Cauchy density is $f(x) = 1/\big(\pi(1 + x^2)\big)$. Find its cumulative distribution function and hence a transformation that turns a uniform deviate U into a Cauchy deviate. Why would averaging Cauchy samples be a bad way to estimate its mean?

$F(x) = \int_{-\infty}^{x} \frac{\mathrm{d}t}{\pi(1+t^2)} = \frac{1}{2} + \frac{1}{\pi}\arctan(x)$. Solving $U = F(X)$ gives $X = \tan\big(\pi(U - \frac{1}{2})\big)$. The Cauchy distribution has such thick tails that its mean and variance do not exist: the sample average never settles down (it is itself Cauchy distributed, however many samples we take). The $1/\sqrt{n}$ rule needs a finite variance.

[4 marks]

(c) We want samples from $f(x) = 6x(1-x)$ on $[0,1]$ using uniform proposals, $g(y) = 1$. Find the smallest c with $cg(x) \geq f(x)$, describe the rejection method, and find the fraction of proposals that are accepted. Explain why the accepted values have density f.

f is largest at $x = \frac{1}{2}$, where $f = \frac{3}{2}$, so $c = 1.5$. Draw Y and U uniformly on $(0,1)$; accept Y if $Ucg(Y) \leq f(Y)$, i.e. with probability $f(Y)/(cg(Y)) = 4Y(1-Y)$; otherwise try again. The density of proposing y and then accepting it is $g(y)f(y)/(cg(y)) = f(y)/c$, which is proportional to f. Integrating, a proposal is accepted with probability $1/c = \frac{2}{3}$.

[5 marks]

(d) To draw samples from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ in d dimensions we use proposals from the wider $\mathcal{N}(\mathbf{0}, s^2\mathbf{I})$ with $s > 1$. Show that the smallest c is s^d. What fraction of proposals is accepted for $s = 1.2$ when $d = 1$, 10 and 100?

The ratio of the densities is

$$\frac{f(\boldsymbol{x})}{g(\boldsymbol{x})} = s^d \mathrm{e}^{-\frac{1}{2}\|\boldsymbol{x}\|^2(1-1/s^2)} \leq s^d,$$

with equality at $\boldsymbol{x} = \mathbf{0}$, so $c = s^d$ and the acceptance rate is s^{-d}: 0.83, 0.16 and 1.2×10^{-8}. Even a slightly wrong proposal is hopeless in high dimensions, which is why we need MCMC.

[3 marks]

(e) In Python, implement the rejection sampler of part (c) with 10^5 proposals. Check the acceptance rate, and that the accepted samples have mean $\frac{1}{2}$ and variance $\frac{1}{20}$.

```
rng = np.random.default_rng(0)
n = 100000
Y = rng.random(n)    # proposals from g = U(0,1)
U = rng.random(n)
x = Y[1.5 * U <= 6 * Y * (1 - Y)]
print(len(x) / n, x.mean(), x.var())
```

This gives an acceptance rate of 0.665 (against $\frac{2}{3}$), mean 0.4998 and variance 0.0496. Working on whole arrays at once is much faster in numpy than a loop that tries again after each rejection.

[4 marks]

End of question 3

Answers

4

(a) M_{ij} is the probability that a Markov chain moves from state j to state i. Explain why $\sum_i M_{ij} = 1$. Show that if $\boldsymbol{\pi}$ satisfies detailed balance, $M_{ij}\pi_j = M_{ji}\pi_i$, then $\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$. What does this mean?

From state j the chain must go somewhere (possibly staying at j), so the probabilities of all destinations sum to 1. Summing detailed balance over j,

$$(\mathbf{M}\boldsymbol{\pi})_i = \sum_j M_{ij}\pi_j = \sum_j M_{ji}\pi_i = \pi_i \sum_j M_{ji} = \pi_i.$$

If the chain's states are distributed according to $\boldsymbol{\pi}$, they still are after another step: $\boldsymbol{\pi}$ is a stationary distribution (an eigenvector of $\mathbf{M}$ with eigenvalue 1).

[4 marks]

(b) Four states sit on a ring, 1–2–3–4–1, with target probabilities $\boldsymbol{\pi} = (0.1, 0.2, 0.3, 0.4)$. From each state the Metropolis algorithm proposes one of its two neighbours, each with probability $\frac{1}{2}$, and accepts with probability $\min(1, \pi_i/\pi_j)$. Write down $\mathbf{M}$ and check that $\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$.

From 1: both neighbours (2 and 4) are more probable, so it moves to each with probability $\frac{1}{2}$. From 2: to 3 with $\frac{1}{2}$, to 1 with $\frac{1}{2} \times \frac{0.1}{0.2} = \frac{1}{4}$, stays with $\frac{1}{4}$. From 3: to 4 with $\frac{1}{2}$, to 2 with $\frac{1}{2} \times \frac{2}{3} = \frac{1}{3}$, stays with $\frac{1}{6}$. From 4: to 3 with $\frac{1}{2} \times \frac{3}{4} = \frac{3}{8}$, to 1 with $\frac{1}{2} \times \frac{1}{4} = \frac{1}{8}$, stays with $\frac{1}{2}$. Each column is a "from" state:

$$\mathbf{M} = \begin{pmatrix} 0 & \frac{1}{4} & 0 & \frac{1}{8} \\ \frac{1}{2} & \frac{1}{4} & \frac{1}{3} & 0 \\ 0 & \frac{1}{2} & \frac{1}{6} & \frac{3}{8} \\ \frac{1}{2} & 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}, \qquad \mathbf{M}\boldsymbol{\pi} = \begin{pmatrix} 0.05 + 0.05 \\ 0.05 + 0.05 + 0.1 \\ 0.1 + 0.05 + 0.15 \\ 0.05 + 0.15 + 0.2 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}.$$

[6 marks]

(c) In Python, find the eigenvalues of $\mathbf{M}$, and starting from state 1, $\boldsymbol{p}(0) = (1,0,0,0)^{\mathsf{T}}$, print $\boldsymbol{p}(t) = \mathbf{M}^t\boldsymbol{p}(0)$ for t up to 10. How quickly does it approach $\boldsymbol{\pi}$?

```
M = np.array([[0, 1/4, 0, 1/8], [1/2, 1/4, 1/3, 0],
              [0, 1/2, 1/6, 3/8], [1/2, 0, 1/2, 1/2]])
print(np.linalg.eigvals(M))
p = np.array([1.0, 0, 0, 0])
for t in range(11):
    print(t, p)
    p = M @ p
```

The eigenvalues are 1, 0.386, 0.052 and -0.521. After 10 steps $\boldsymbol{p} = (0.100, 0.199, 0.301, 0.399)$. The error is dominated by the eigenvalue -0.521: it roughly halves each step and changes sign, so $\boldsymbol{p}(t)$ overshoots back and forth as it converges.

[3 marks]

Answers

(d) To sample from a posterior $\pi(\boldsymbol{\theta}) = f(\mathcal{D}|\boldsymbol{\theta})f(\boldsymbol{\theta})/f(\mathcal{D})$ the Metropolis algorithm accepts a proposal $\boldsymbol{\theta}'$ with probability $\min(1, \pi(\boldsymbol{\theta}')/\pi(\boldsymbol{\theta}))$. Why don't we need $f(\mathcal{D})$? Why do programs compute $\log\pi(\boldsymbol{\theta}') - \log\pi(\boldsymbol{\theta})$ rather than the ratio itself?

$f(\mathcal{D})$ appears in both $\pi(\boldsymbol{\theta}')$ and $\pi(\boldsymbol{\theta})$ and cancels from the ratio, so we only need likelihood times prior. The likelihood is a product of many probabilities and soon underflows to 0 in floating point; its log is a sum and causes no trouble. (Accept if $\log U < \log\pi(\boldsymbol{\theta}') - \log\pi(\boldsymbol{\theta})$.)

[3 marks]

(e) In Python, use Metropolis with proposals $x' = x + s\epsilon$, $\epsilon \sim \mathcal{N}(0,1)$, to sample from $\pi(x) \propto \mathrm{e}^{-(x^2-4)^2/8}$, which has peaks at $x = \pm 2$. Start at $x = 2$ and run 10^5 steps for $s = 0.1$, 1 and 10. For each, print the acceptance rate, the fraction of samples with $x > 0$ (which should be $\frac{1}{2}$), the number of times x changes sign, and the estimate of $\mathbb{E}[x^2]$ (whose exact value is 3.41). Comment.

```
rng = np.random.default_rng(0)
log_pi = lambda x: -(x**2 - 4)**2 / 8
for s in [0.1, 1.0, 10.0]:
    x, xs, acc = 2.0, [], 0
    for t in range(100000):
        y = x + s * rng.standard_normal()
        if np.log(rng.random()) < log_pi(y) - log_pi(x):
            x, acc = y, acc + 1
        xs.append(x)    # repeats count when we stay
    xs = np.array(xs)
    sg = np.sign(xs)
    crossings = np.sum(sg[1:] != sg[:-1])
    print(s, acc / len(xs), np.mean(xs > 0))
    print(crossings, np.mean(xs**2))
```

s	accepted	$x > 0$	sign changes	$\mathbb{E}[x^2]$
0.1	0.95	0.61	221	3.47
1	0.59	0.51	4533	3.41
10	0.15	0.49	7257	3.42

With $s = 0.1$ almost every move is accepted, but the chain creeps and rarely crosses the dip at $x = 0$, so it spends too long in the peak where it started. With $s = 10$ most proposals are rejected, but those accepted often jump straight between the peaks. A step size comparable with the width of π works well.

[4 marks]

End of question 4

Answers

5 This question is harder and optional. It shows why EM never decreases the likelihood, why an MCMC chain with detailed balance settles down, and runs alternating least squares on the lecture's ratings.

(a) For data $\mathcal{D}$, latent variables $\boldsymbol{Z}$ and parameters $\boldsymbol{\Theta}$, show that for every $\boldsymbol{Z}$

$$\log f(\mathcal{D}|\boldsymbol{\Theta}) = \log f(\mathcal{D},\boldsymbol{Z}|\boldsymbol{\Theta}) - \log \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta}).$$

Average both sides over $P_t(\boldsymbol{Z}) = \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta}^{(t)})$ to get $\log f(\mathcal{D}|\boldsymbol{\Theta}) = Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) + S(\boldsymbol{\Theta})$, where Q is the function EM maximises and $S(\boldsymbol{\Theta}) = -\sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta})$. Using $\log(x) \le x - 1$, show that $S(\boldsymbol{\Theta}) \ge S(\boldsymbol{\Theta}^{(t)})$ for every $\boldsymbol{\Theta}$, and hence that an EM step cannot decrease the likelihood.

The identity is $f(\mathcal{D},\boldsymbol{Z}|\boldsymbol{\Theta}) = \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta})\, f(\mathcal{D}|\boldsymbol{\Theta})$ (the definition of conditional probability) after taking logs. The left-hand side does not depend on $\boldsymbol{Z}$ and $\sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) = 1$, so averaging gives

$$\log f(\mathcal{D}|\boldsymbol{\Theta}) = \underbrace{\sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log f(\mathcal{D},\boldsymbol{Z}|\boldsymbol{\Theta})}_{Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})} \underbrace{- \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta})}_{S(\boldsymbol{\Theta})}.$$

Writing $P(\boldsymbol{Z}) = \mathbb{P}(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta})$,

$$S(\boldsymbol{\Theta}^{(t)}) - S(\boldsymbol{\Theta}) = \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log \frac{P(\boldsymbol{Z})}{P_t(\boldsymbol{Z})} \le \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \left(\frac{P(\boldsymbol{Z})}{P_t(\boldsymbol{Z})} - 1 \right) = 1 - 1 = 0.$$

(The difference $S(\boldsymbol{\Theta}) - S(\boldsymbol{\Theta}^{(t)})$ is the KL divergence between P_t and P.) The M-step chooses $\boldsymbol{\Theta}^{(t+1)}$ to maximise Q, so $Q(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) \ge Q(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)})$, and S cannot go down either, so $\log f(\mathcal{D}|\boldsymbol{\Theta}^{(t+1)}) \ge \log f(\mathcal{D}|\boldsymbol{\Theta}^{(t)})$.

[7 marks]

(b) Let $\mathbf{D} = \operatorname{diag}(\boldsymbol{\pi})$ with all $\pi_i > 0$. Show that if $\mathbf{M}$ satisfies detailed balance then $\mathbf{A} = \mathbf{D}^{-1/2}\mathbf{M}\mathbf{D}^{1/2}$ is symmetric. Deduce that the eigenvalues of $\mathbf{M}$ are real and that $\mathbf{M}$ has a full set of eigenvectors (the assumption made in the lecture when we expanded $\boldsymbol{p}(0)$ in eigenvectors). Check that $A_{12} = A_{21}$ for the matrix of Question 4(b).

$A_{ij} = \pi_i^{-1/2} M_{ij} \pi_j^{1/2}$ and $A_{ji} = \pi_j^{-1/2} M_{ji} \pi_i^{1/2}$. Multiplying both by $\sqrt{\pi_i \pi_j}$ gives $M_{ij}\pi_j$ and $M_{ji}\pi_i$, which are equal by detailed balance, so $A_{ij} = A_{ji}$. A real symmetric matrix has real eigenvalues and an orthonormal set of eigenvectors $\boldsymbol{w}_k$. If $\mathbf{A}\boldsymbol{w} = \lambda \boldsymbol{w}$ then $\mathbf{M}(\mathbf{D}^{1/2}\boldsymbol{w}) = \mathbf{D}^{1/2}\mathbf{A}\boldsymbol{w} = \lambda \mathbf{D}^{1/2}\boldsymbol{w}$, so $\mathbf{M}$ has the same eigenvalues, with independent eigenvectors $\mathbf{D}^{1/2}\boldsymbol{w}_k$. For Question 4(b), $A_{12} = \frac{1}{4}\sqrt{0.2/0.1} = \frac{\sqrt{2}}{4}$ and $A_{21} = \frac{1}{2}\sqrt{0.1/0.2} = \frac{1}{2\sqrt{2}} = \frac{\sqrt{2}}{4}$.

[6 marks]

Answers

(c) In matrix factorisation with **B** fixed, row $\boldsymbol{a}_i$ of **A** minimises $\sum_j (X_{ij} - \boldsymbol{a}_i^\mathsf{T}\boldsymbol{b}_j)^2 + \eta\|\boldsymbol{a}_i\|^2$, the sum running over the films user i has rated. Differentiate to show that $\boldsymbol{a}_i = \left(\eta\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\right)^{-1}\sum_j \boldsymbol{b}_j X_{ij}$, and check that it reduces to the formula of Question 2(d) when $K = 1$. Why is the matrix being inverted always invertible when $\eta > 0$?

The gradient is $-2\sum_j \boldsymbol{b}_j(X_{ij} - \boldsymbol{b}_j^\mathsf{T}\boldsymbol{a}_i) + 2\eta\boldsymbol{a}_i$ (using $\boldsymbol{a}_i^\mathsf{T}\boldsymbol{b}_j = \boldsymbol{b}_j^\mathsf{T}\boldsymbol{a}_i$). Setting it to zero gives $\left(\eta\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\right)\boldsymbol{a}_i = \sum_j \boldsymbol{b}_j X_{ij}$. For $K = 1$ this is $(\eta + \sum_j b_j^2)a = \sum_j b_j X_j$. For any $\boldsymbol{v} \neq \mathbf{0}$, $\boldsymbol{v}^\mathsf{T}\left(\eta\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\right)\boldsymbol{v} = \eta\|\boldsymbol{v}\|^2 + \sum_j(\boldsymbol{b}_j^\mathsf{T}\boldsymbol{v})^2 > 0$, so the matrix is positive definite and hence invertible, even if the user has rated fewer than K films.

[5 marks]

(d) In Python, run alternating least squares with $K = 1$ and $\eta = 0.1$ on the ratings of Question 2(b) (use `np.nan` for the missing entries), starting from random factors. Compare the predictions for Bob's rating of The Matrix and Chas's rating of Titanic with your answers to Question 2(b).

```
R = np.array([[2, np.nan, 4, 5, 2],
              [5, 3, np.nan, 3, np.nan],
              [np.nan, 2, 3, 5, 3]])
W = np.logical_not(np.isnan(R))
K, eta = 1, 0.1
rng = np.random.default_rng(0)
A, B = rng.random((3, K)) + 1, rng.random((K, 5)) + 1
for it in range(200):
    for i in range(3):    # ridge for row i of A
        Bi, Ri = B[:, W[i]], R[i, W[i]]
        G = eta * np.eye(K) + Bi @ Bi.T
        A[i] = np.linalg.solve(G, Bi @ Ri)
    for j in range(5):    # ridge for column j of B
        Aj, Rj = A[W[:, j]], R[W[:, j], j]
        G = eta * np.eye(K) + Aj.T @ Aj
        B[:, j] = np.linalg.solve(G, Aj.T @ Rj)
print(np.round(A @ B, 2))
```

This predicts 3.52 for Bob and The Matrix and 3.66 for Chas and Titanic, against 3.80 and 3.39 from the means. Other random starts give the same answer. With only 11 ratings and $3 + 5 = 8$ parameters there is little to learn: without subtracting the means, a rank-1 model mostly learns how highly each film is rated overall (Spartacus highest, Dr No and Ten lowest), and the training RMS error is still 0.88 because Alice and Bob disagree about Titanic. Matrix factorisation pays off on large, sparse data sets.

[7 marks]

End of question 5

END OF PAPER